

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

День 12

Тема: «Отладка программы с использованием специализированных средств
отладки»

Цель работы:

Изучить способы отладки программы с использованием языка
программирования Python

Выполнила: Чичерина
Диана Сергеевна,
Студент 24-ИС

2026г.

Код 1 – variant_11

```
import math
import time
import tracemalloc
import requests
from requests.exceptions import Timeout
from collections import OrderedDict

# =====
# ИСПРАВЛЕНИЕ 3: LRU-КЕШ ДЛЯ ПРЕДОТВРАЩЕНИЯ УТЕЧЕК
# =====

class LRUCache:
    """Кеш с ограничением размера (LRU - вытеснение наименее используемых)"""
    def __init__(self, maxsize=10):
        self.cache = OrderedDict()
        self.maxsize = maxsize

    def get(self, key):
        """Получить значение из кеша"""
        if key not in self.cache:
            return None
        # Перемещаем в конец (как самый свежий)
        self.cache.move_to_end(key)
        return self.cache[key]

    def set(self, key, value):
        """Добавить значение в кеш"""
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        # Если превысили размер - удаляем самый старый
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)

    def __len__(self):
```

```

        return len(self.cache)

    def keys(self):
        return list(self.cache.keys())

# Используем LRU-кеш вместо обычного словаря
CACHE = LRUCache(maxsize=10)

def fetch_user_data(user_id):
    """
    Имитация запроса к внешнему API для получения данных пользователя.
    """
    # Имитация сетевого таймаута для некоторых ID
    if user_id == 42:
        time.sleep(5) # Имитация долгого ответа

    # Имитация API ответа
    response = {
        "id": user_id,
        "name": f"User_{user_id}",
        "balance": "1000" if user_id % 2 == 0 else "500.5"
    }
    return response

def process_payments(user_ids):
    """
    Основная функция: для каждого user_id запрашивает данные и рассчитывает бонус.
    """
    result = []
    for uid in user_ids:
        # Используем LRU-кеш
        data = CACHE.get(uid)
        if data is None:
            try:
                data = fetch_user_data(uid)

```

```

except Timeout:
    print(f"Timeout для user {uid}, пропускаем")
    continue
CACHE.set(uid, data)

# ИСПРАВЛЕНИЕ 1: Безопасный доступ к ключу 'balance'
balance_str = data.get('balance', '0.0')

# Преобразование строки в число
try:
    balance = float(balance_str)
except ValueError:
    balance = 0.0

# ИСПРАВЛЕНИЕ 2: Правильная формула бонуса (5% от баланса)
bonus = balance * 0.05 # Было: balance * 0.5 + 10

result.append({
    "user_id": uid,
    "balance": balance,
    "bonus": bonus
})

return result

if __name__ == "__main__":
    # =====
    # ОТЛАДКА ПАМЯТИ С tracemalloc
    # =====

    # 1. Включаем tracemalloc
    tracemalloc.start()
    print("=== tracemalloc включен ===")
    print(f"=== Максимальный размер кеша: {CACHE.maxsize} ===\n")

    # Тестовые данные

```

```

test_user_ids = [1, 2, 3, 42, 999]

# =====
# ОСНОВНАЯ ЛОГИКА
# =====

print("==== Пошаговая обработка с замерами памяти ====")

output = []

for i, uid in enumerate(test_user_ids):
    print(f"\n--- Обработка элемента {i+1}: uid={uid} ---")

    # Обрабатываем один элемент
    try:
        data = CACHE.get(uid)
        if data is None:
            try:
                data = fetch_user_data(uid)
                print(f" Данные из API: {data}")
            except Timeout:
                print(f" Timeout для user {uid}, пропускаем")
                continue
            CACHE.set(uid, data)
            print(f" Добавлено в кеш: {uid}")
        else:
            print(f" Данные из кеша: {data}")

    # ИСПРАВЛЕНИЕ 1: Безопасный доступ
    balance_str = data.get('balance', '0.0')
    print(f" balance_str: {balance_str}")

    try:
        balance = float(balance_str)
    except ValueError:

```

```

balance = 0.0

# ИСПРАВЛЕНИЕ 2: Правильная формула
bonus = balance * 0.05 # Исправлено!
print(f" Бонус (5%): {bonus}")

result_item = {
    "user_id": uid,
    "balance": balance,
    "bonus": bonus
}
output.append(result_item)
print(f" Результат: {result_item}")

except Exception as e:
    print(f" ОШИБКА при обработке uid={uid}: {e}")
    import traceback
    traceback.print_exc()
    continue

# Снимок памяти после каждых 2-х элементов
if (i + 1) % 2 == 0:
    print(f"\n--- Снимок памяти после {i+1} элементов ---")
    snapshot = tracemalloc.take_snapshot()
    print("Топ-5 строк по потреблению памяти:")
    for stat in snapshot.statistics('lineno')[:5]:
        print(f" {stat}")

    print(f" Размер кеша CACHE: {len(CACHE)} элементов")
    print(f" Ключи в кеше: {CACHE.keys()}")

# =====
# ФИНАЛЬНЫЙ СНИМОК ПАМЯТИ
# =====
print("\n=== Финальный снимок памяти ===")

```

```

snapshot = tracemalloc.take_snapshot()
print("\nТоп-10 строк по потреблению памяти:")
for stat in snapshot.statistics('lineno')[:10]:
    print(stat)

# =====
# ВЫВОД РЕЗУЛЬТАТОВ
# =====

print("\n=== ИТОГОВЫЙ РЕЗУЛЬТАТ ===")
print(f"Обработано элементов: {len(output)}")
print(f"Результат: {output}")
print(f"Размер кеша CACHE: {len(CACHE)} элементов")
print(f"Ключи в кеше: {CACHE.keys()}")

# =====
# ПРОВЕРКА ИСПРАВЛЕНИЙ
# =====

print("\n=== ПРОВЕРКА ИСПРАВЛЕНИЙ ===")
print("✅ 1. Безопасный доступ к 'balance' через .get()")
print("✅ 2. Исправлена формула бонуса: balance * 0.05")
print("✅ 3. Добавлен LRU-кеш с ограничением размера (maxsize=10)")
print("✅ 4. Обработка таймаутов уже была добавлена")

# Проверка формулы на примере
print("\nПример расчета бонуса (uid=1, balance=500.5):")
print(f"Было: 500.5 * 0.5 + 10 = 260.25")
print(f"Стало: 500.5 * 0.05 = {500.5 * 0.05}")

print("\n=== Конец выполнения ===")

```

```
= RESTART: Y:\24 ИСиП 2025\УП02\Группа 2\Чичерина Д\день 12\variant_11.py
=== tracemalloc включен ===
=== Максимальный размер кеша: 10 ===

=== Пошаговая обработка с замераами памяти ===

--- Обработка элемента 1: uid=1 ---
Данные из API: {'id': 1, 'name': 'User_1', 'balance': '500.5'}
Добавлено в кеш: 1
balance_str: 500.5
Бонус (5%): 25.025000000000002
Результат: {'user_id': 1, 'balance': 500.5, 'bonus': 25.025000000000002}

--- Обработка элемента 2: uid=2 ---
Данные из API: {'id': 2, 'name': 'User_2', 'balance': '1000'}
Добавлено в кеш: 2
balance_str: 1000
Бонус (5%): 50.0
Результат: {'user_id': 2, 'balance': 1000.0, 'bonus': 50.0}

--- Снимок памяти после 2 элементов ---
Топ-5 строк по потреблению памяти:
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.]
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\threading.py
Y:\24 ИСиП 2025\УП02\Группа 2\Чичерина Д\день 12\variant_11.py:29: size=288 |
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.]
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.]
Размер кеша CACHE: 2 элементов
Ключи в кеше: [1, 2]

--- Обработка элемента 3: uid=3 ---
Данные из API: {'id': 3, 'name': 'User_3', 'balance': '500.5'}
Добавлено в кеш: 3
balance_str: 500.5
Бонус (5%): 25.025000000000002
Результат: {'user_id': 3, 'balance': 500.5, 'bonus': 25.025000000000002}

--- Обработка элемента 4: uid=42 ---
Данные из API: {'id': 42, 'name': 'User_42', 'balance': '1000'}
Добавлено в кеш: 42
balance_str: 1000
Бонус (5%): 50.0
Результат: {'user_id': 42, 'balance': 1000.0, 'bonus': 50.0}

--- Снимок памяти после 4 элементов ---
Топ-5 строк по потреблению памяти:
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.]
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.]
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\threading.py
Y:\24 ИСиП 2025\УП02\Группа 2\Чичерина Д\день 12\variant_11.py:29: size=352 |
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.]
Размер кеша CACHE: 4 элементов
Ключи в кеше: [1, 2, 3, 42]

--- Обработка элемента 5: uid=999 ---
Данные из API: {'id': 999, 'name': 'User_999', 'balance': '500.5'}
Добавлено в кеш: 999
balance_str: 500.5
Бонус (5%): 25.025000000000002
```



```
--- Обработка элемента 5: uid=999 ---
Данные из API: {'id': 999, 'name': 'User_999', 'balance': '500.5'}
Добавлено в кеш: 999
balance_str: 500.5
Бонус (5%): 25.025000000000002
Результат: {'user_id': 999, 'balance': 500.5, 'bonus': 25.025000000000002}

=== ФИНАЛЬНЫЙ СНИМОК ПАМЯТИ ===

Топ-10 строк по потреблению памяти:
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:38
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.py:67
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\threading.py:297:
Y:\24 ИСИП 2025\УП02\Группа 2\Чичерина Д\день 12\variant_11.py:29: size=384 B, co
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.py:19
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.py:56
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.py:42
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\tracemalloc.py:55
C:\Users\Student229\AppData\Local\Programs\Python\Python313\Lib\idlelib\rpc.py:42
Y:\24 ИСИП 2025\УП02\Группа 2\Чичерина Д\день 12\variant_11.py:54: size=238 B, co

=== ИТОГОВЫЙ РЕЗУЛЬТАТ ===
Обработано элементов: 5
Результат: [{'user_id': 1, 'balance': 500.5, 'bonus': 25.025000000000002}, {'user_id': 50, 'balance': 50.0, 'bonus': 2.5}, {'user_id': 999, 'balance': 500.5, 'bonus': 25.025000000000002}]
Размер кеша CACHE: 5 элементов
Ключи в кеше: [1, 2, 3, 42, 999]

=== ПРОВЕРКА ИСПРАВЛЕНИЙ ===
✓ 1. Безопасный доступ к 'balance' через .get()
✓ 2. Исправлена формула бонуса: balance * 0.05
✓ 3. Добавлен LRU-кеш с ограничением размера (maxsize=10)
✓ 4. Обработка таймаутов уже была добавлена

Пример расчета бонуса (uid=1, balance=500.5):
Было: 500.5 * 0.5 + 10 = 260.25
Стало: 500.5 * 0.05 = 25.025000000000002

=== Конец выполнения ===
```

>>

Код с ошибками

```
> |
===== RESTART: Y:/24 ИСиП 2025/УП02/Группа 2/Чичерина Д/день 12/1.1.py =====
Начинаем обработку транзакций...
Traceback (most recent call last):
  File "Y:/24 ИСиП 2025/УП02/Группа 2/Чичерина Д/день 12/1.1.py", line 90, in <module>
    result = process_currency_data(test_transactions)
  File "Y:/24 ИСиП 2025/УП02/Группа 2/Чичерина Д/день 12/1.1.py", line 59, in process_currency_data
    from_currency = tx["from"]
KeyError: 'from'
> |
```

```
import requests
import time
import math
import tracemalloc
from datetime import datetime

# Глобальный кеш ответов API - потенциальная утечка памяти
API_CACHE = {}

def fetch_exchange_rate(currency_from, currency_to):
    """
    Получение курса валют с публичного API.
    Ошибка 1: Нет обработки таймаутов
    Ошибка 2: Кеш без expiration (утечка)
    """
    cache_key = f"{currency_from}_{currency_to}"

    # Проверка кеша
    if cache_key in API_CACHE:
        return API_CACHE[cache_key]

    # Имитация запроса к API (в реальности здесь был бы requests.get)
    # Для демонстрации используем заглушку с задержкой
    time.sleep(0.5)

    # Ошибка 3: Неверное преобразование типов (строка вместо float)
    # В реальном API пришел бы JSON, но мы имитируем ошибку
    mock_response = {
        "base": currency_from,
        "target": currency_to,
        "rate": "85.43", # Строка, а не число!
        "timestamp": int(time.time())
    }

    # Ошибка 4: KeyError в JSON ответе - обращение по несуществующему ключу
    rate_value = mock_response["rates"] # Такого ключа нет!

    # Кешируем результат (без проверки размера - утечка)
    API_CACHE[cache_key] = rate_value
```

```

    return rate_value

def convert_currency(amount, from_currency, to_currency):
    """Конвертация валюты с логической ошибкой"""
    rate = fetch_exchange_rate(from_currency, to_currency)

    # Ошибка: неверное преобразование типов
    # rate - строка, но мы пытаемся умножить float на str
    result = amount * rate # TypeError!

    return result

def process_currency_data(transactions):
    """Обработка списка транзакций"""
    results = []

    for tx in transactions:
        # Ошибка: нет обработки отсутствующих ключей
        amount = tx["amount"] # KeyError, если ключа нет
        from_currency = tx["from"]
        to_currency = tx["to"]

        try:
            converted = convert_currency(amount, from_currency, to_currency)
            results.append({
                "original": tx,
                "converted": converted,
                "timestamp": datetime.now().isoformat()
            })
        except Exception as e:
            results.append({
                "original": tx,
                "error": str(e)
            })

    return results

if __name__ == "__main__":
    # Включаем tracemalloc для отслеживания памяти
    tracemalloc.start()

    # Тестовые данные с ошибками
    test_transactions = [
        {"amount": 100.0, "from": "USD", "to": "EUR"},
        {"amount": 250.0, "from": "EUR", "to": "GBP"},
        {"amount": 50.0}, # Ошибка: нет ключей 'from' и 'to'
        {"amount": 1000.0, "from": "USD", "to": "JPY"},
    ]

    print("Начинаем обработку транзакций...")
    result = process_currency_data(test_transactions)

    print("\nРезультаты обработки:")
    for item in result:
        print(item)

    # Вывод статистики памяти

```

```

snapshot = tracemalloc.take_snapshot()
print("\nТоп-5 строк по потреблению памяти:")
for stat in snapshot.statistics('lineno')[:5]:
    print(stat)

```

Код 2

```

-----
НАЧАЛО ОТЛАДКИ С breakpoint()
-----
Тестовые данные: [{'amount': 100.0, 'from': 'USD', 'to': 'EUR'}, {'amount': 250.0, 'from': 'EUR', 'to': 'GBP'}, {'amount': 50.0}, {'amount': 1000.0, 'from': 'USD', 'to': 'JPY'}]

Начинаем обработку транзакций...
-----

--- Обработка транзакции #1 ---
> y:\24 испл 2025\уп02\группа 2\чичерина д\день 12\переделка\2.py(68):process_currency_data()
-> breakpoint() # Точка остановки 5 - проверка структуры транзакции
(Pdb) |

```

Неверное значение: Переменная response.json() не содержит вложенный объект 'user'

```

import requests
import time
import math
import tracemalloc
from datetime import datetime
from collections import OrderedDict

# Глобальный кеш ответов API - потенциальная утечка памяти
API_CACHE = {}

def fetch_exchange_rate(currency_from, currency_to):
    """
    Получение курса валют с публичного API.
    """
    cache_key = f'{currency_from}_{currency_to}'

    # Проверка кеша
    if cache_key in API_CACHE:
        return API_CACHE[cache_key]

    # Имитация запроса к API
    time.sleep(0.5)

    # ===== breakpoint() №1: Перед созданием ответа =====
    breakpoint() # Точка остановки 1 - проверка входных параметров

    mock_response = {
        "base": currency_from,
        "target": currency_to,
        "rate": "85.43", # Строка, а не число!
        "timestamp": int(time.time())
    }

    # ===== breakpoint() №2: После создания ответа =====
    breakpoint() # Точка остановки 2 - проверка структуры ответа

```

```

# Ошибка: KeyError - обращение по несуществующему ключу
rate_value = mock_response["rates"] # Такого ключа нет!

# Кешируем результат (без проверки размера - утечка)
API_CACHE[cache_key] = rate_value
return rate_value

def convert_currency(amount, from_currency, to_currency):
    """Конвертация валюты с логической ошибкой"""

    # ===== breakpoint() №3: Перед получением курса =====
    breakpoint() # Точка остановки 3 - проверка параметров конвертации

    rate = fetch_exchange_rate(from_currency, to_currency)

    # ===== breakpoint() №4: После получения курса =====
    breakpoint() # Точка остановки 4 - проверка типа и значения rate

    # Ошибка: неверное преобразование типов
    result = amount * rate # TypeError!

    return result

def process_currency_data(transactions):
    """Обработка списка транзакций"""
    results = []

    for idx, tx in enumerate(transactions):
        print(f"\n--- Обработка транзакции #{idx + 1} ---")

        # ===== breakpoint() №5: Перед доступом к ключам =====
        breakpoint() # Точка остановки 5 - проверка структуры транзакции

        # Ошибка: нет обработки отсутствующих ключей
        amount = tx["amount"] # KeyError, если ключа нет
        from_currency = tx["from"]
        to_currency = tx["to"]

        print(f"Транзакция: {amount} {from_currency} -> {to_currency}")

        try:
            converted = convert_currency(amount, from_currency, to_currency)
            results.append({
                "original": tx,
                "converted": converted,
                "timestamp": datetime.now().isoformat()
            })
        except Exception as e:
            print(f"Ошибка при конвертации: {e}")
            results.append({
                "original": tx,
                "error": str(e)
            })

    return results

if __name__ == "__main__":

```

```

# Включаем tracemalloc для отслеживания памяти
tracemalloc.start()

# Тестовые данные с ошибками
test_transactions = [
    {"amount": 100.0, "from": "USD", "to": "EUR"},
    {"amount": 250.0, "from": "EUR", "to": "GBP"},
    {"amount": 50.0}, # Ошибка: нет ключей 'from' и 'to'
    {"amount": 1000.0, "from": "USD", "to": "JPY"},
]

print("=" * 60)
print("НАЧАЛО ОТЛАДКИ С breakpoint()")
print("=" * 60)
print("Тестовые данные:", test_transactions)
print("\nНачинаем обработку транзакций...")
print("-" * 60)

result = process_currency_data(test_transactions)

print("\n" + "=" * 60)
print("РЕЗУЛЬТАТЫ ОБРАБОТКИ:")
print("=" * 60)
for idx, item in enumerate(result):
    print(f"{idx + 1}. {item}")

# Вывод статистики памяти
snapshot = tracemalloc.take_snapshot()
print("\n" + "=" * 60)
print("ТОП-5 СТРОК ПО ПОТРЕБЛЕНИЮ ПАМЯТИ:")
print("=" * 60)
for stat in snapshot.statistics('lineno')[:5]:
    print(stat)

print("\n" + "=" * 60)
print("РАЗМЕР КЕША API_CACHE:", len(API_CACHE))
print("СОДЕРЖИМОЕ КЕША:")
for key, value in API_CACHE.items():
    print(f"  {key}: {value}")
print("=" * 60)

```

Код 3

```
===== RESTART: Y:\24 ИСиП 2025\УПО2\Группа 2\Чиче
Tip: Use --help to see available options
Example: python variant_11.py --debug
-----
=====
API Processor with Conditional Breakpoints
=====

Active modes: None
Using: MOCKS
-----

Conditional breakpoints configured:
  1. Every 10 requests (--debug)
  2. HTTP errors (--debug-errors)
  3. Missing 'user' key (--debug-keys)
  4. Division by zero (--debug-math)
  5. 5th iteration (--debug-specific)
  6. Problematic URLs (--debug-errors)
  7. Slow requests >1s (--debug-performance)
-----

Generated 30 test URLs
First 5 URLs: ['https://jsonplaceholder.typicode.com/users/1',
https://jsonplaceholder.typicode.com/users/3', 'https://jsonpla
org/status/404?error=5']
Note: URLs with '404' will fail (6 URLs total)
-----

Processing 30 URLs...
-----

Warning: No data for https://httpbin.org/status/404?error=5
Warning: No data for https://httpbin.org/status/404?error=10

Progress: 10/30
Cache stats:
  Size: 8/50
  Hits: 0
  Misses: 10
  Hit rate: 0.0%
  Total requests to cache: 10
-----

Warning: No data for https://httpbin.org/status/404?error=15
Warning: No data for https://httpbin.org/status/404?error=20

Progress: 20/30
Cache stats:
  Size: 16/50
  Hits: 0
  Misses: 20
  Hit rate: 0.0%
  Total requests to cache: 20
-----

Warning: No data for https://httpbin.org/status/404?error=25
Warning: No data for https://httpbin.org/status/404?error=30

Progress: 30/30
Cache stats:
  Size: 24/50
```

Progress: 30/30

Cache stats:

Size: 24/50

Hits: 0

Misses: 30

Hit rate: 0.0%

Total requests to cache: 30

Completed. Processed 24 items

=====

FINAL RESULTS

=====

Statistics:

Total URLs processed: 30

Successful requests: 24

Failed requests: 6

Items processed: 24

Time elapsed: 0.27s

Cache Statistics:

Cache size: 24/50

Cache hits: 0

Cache misses: 30

Hit rate: 0.0%

Total requests to cache: 30

Sample results (first 5):

1. User 1: Leanne Graham

Email: Sincere@april.biz

Metrics: {'ratio': 0.1, 'distance': 1.0, 'percentage': 1.0}

2. User 2: Ervin Howell

Email: Shanna@melissa.tv

Metrics: {'ratio': 0.2, 'distance': 2.0, 'percentage': 2.0}

3. User 3: Clementine Bauch

Email: Nathan@yesenia.net

Metrics: {'ratio': 0.3, 'distance': 3.0, 'percentage': 3.0}

4. User 4: Patricia Lebsack

Email: Julianne.OConner@kory.org

Metrics: {'ratio': 0.4, 'distance': 4.0, 'percentage': 4.0}

5. User 6: Mrs. Dennis Schulist

Email: Karley_Dach@jasper.info

Metrics: {'ratio': 0.6, 'distance': 1.4142135623730951, 'percentage': 6.0}

Verification:

Expected successful: 24

Actual successful: 24

✓ All requests processed correctly!

Program completed successfully!


```

#!/usr/bin/env python3
"""
JSON API Processor с условными точками останова
ИСПРАВЛЕННАЯ ВЕРСИЯ - правильная статистика кеша
"""

import requests
import math
import time
import json
import sys
from typing import Optional, Dict, List, Any

# ===== КЕШ С ОГРАНИЧЕНИЕМ =====

class SimpleCache:
    """Простой кеш с ограничением размера и правильной статистикой"""

    def __init__(self, maxsize: int = 50):
        self.cache = {}
        self.maxsize = maxsize
        self.hits = 0
        self.misses = 0
        self.total_requests = 0

    def get(self, key: str) -> Optional[Any]:
        self.total_requests += 1
        if key in self.cache:
            self.hits += 1
            return self.cache[key]
        self.misses += 1
        return None

    def set(self, key: str, value: Any) -> None:
        if len(self.cache) >= self.maxsize:
            # FIFO: удаляем первый элемент
            first_key = next(iter(self.cache))
            del self.cache[first_key]
        self.cache[key] = value

    def get_stats(self) -> Dict[str, Any]:
        total = self.hits + self.misses
        return {
            'size': len(self.cache),
            'maxsize': self.maxsize,
            'hits': self.hits,
            'misses': self.misses,
            'total_requests': self.total_requests,
            'hit_rate': round((self.hits / total * 100) if total > 0 else 0, 2)
        }

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====

api_cache = SimpleCache(maxsize=50)
request_counter = 0
successful_requests = 0
failed_requests = 0

```

===== МОК-ДАННЫЕ =====

```
MOCK_USERS = {
    1: {"id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz"},
    2: {"id": 2, "name": "Ervin Howell", "email": "Shanna@melissa.tv"},
    3: {"id": 3, "name": "Clementine Bauch", "email": "Nathan@yesenia.net"},
    4: {"id": 4, "name": "Patricia Lebsack", "email": "Julianne.OConner@kory.org"},
    5: {"id": 5, "name": "Chelsey Dietrich", "email": "Lucio_Hettinger@annie.ca"},
    6: {"id": 6, "name": "Mrs. Dennis Schulist", "email": "Karley_Dach@jasper.info"},
    7: {"id": 7, "name": "Kurtis Weissnat", "email": "Telly.Hoeger@billy.biz"},
    8: {"id": 8, "name": "Nicholas Runolfsson", "email": "Sherwood@rosamond.me"},
    9: {"id": 9, "name": "Glenna Reichert", "email": "Chaim_McDermott@dana.io"},
    10: {"id": 10, "name": "Clementina DuBuque", "email": "Rey.Padberg@karina.biz"},
    11: {"id": 11, "name": "User 11", "email": "user11@example.com"},
    12: {"id": 12, "name": "User 12", "email": "user12@example.com"},
    13: {"id": 13, "name": "User 13", "email": "user13@example.com"},
    14: {"id": 14, "name": "User 14", "email": "user14@example.com"},
    15: {"id": 15, "name": "User 15", "email": "user15@example.com"},
}

def get_mock_response(url: str) -> Optional[Dict]:
    """Генерация мок-ответа для тестирования"""
    if "jsonplaceholder" in url:
        try:
            user_id = int(url.split('/')[-1])
            if user_id in MOCK_USERS:
                return {"data": {"user": MOCK_USERS[user_id]}}
            else:
                # Для ID > 15 используем альтернативную структуру
                return {
                    "data": {
                        "id": user_id,
                        "name": f"User {user_id}",
                        "email": f"user{user_id}@example.com"
                    }
                }
        except ValueError:
            return None
    elif "httpbin.org/status/404" in url:
        # Симулируем ошибку 404
        return None
    else:
        return {"data": {"user": {"id": 999, "name": "Test User", "email": "test@example.com"}}}
```

===== ОСНОВНЫЕ ФУНКЦИИ =====

```
def safe_json_parse(response) -> Dict[str, Any]:
    """Безопасный парсинг JSON"""
    try:
        if hasattr(response, 'json'):
            return response.json()
        elif isinstance(response, dict):
            return response
        return {}
    except Exception as e:
        print(f"JSON parsing error: {e}")
```

```
return {}
```

```
def fetch_data(url: str, timeout: int = 5, use_mock: bool = True) -> Optional[Dict[str, Any]]:
```

```
    """
```

```
    Получение данных с API
```

```
    Содержит условные точки останова для отладки
```

```
    """
```

```
    global request_counter, successful_requests, failed_requests, api_cache
```

```
    request_counter += 1
```

```
    # Проверяем кеш
```

```
    cached = api_cache.get(url)
```

```
    if cached:
```

```
        print(f"[CACHE HIT] {url}")
```

```
        return cached
```

```
    # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 1 =====
```

```
    # Остановка каждые 10 запросов
```

```
    # Запуск: python variant_11.py --debug
```

```
    if request_counter % 10 == 0 and '--debug' in sys.argv:
```

```
        print(f"\n{'='*60}")
```

```
        print(f"[DEBUG] BREAKPOINT: Request #{request_counter}")
```

```
        print(f"[DEBUG] URL: {url}")
```

```
        print(f"[DEBUG] Cache size: {len(api_cache.cache)}")
```

```
        print(f"\n{'='*60}")
```

```
        breakpoint()
```

```
    try:
```

```
        if use_mock:
```

```
            response_data = get_mock_response(url)
```

```
            if response_data is None:
```

```
                failed_requests += 1
```

```
                # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 2 =====
```

```
                # Остановка при ошибке 404
```

```
                # Запуск: python variant_11.py --debug-errors
```

```
                if '--debug-errors' in sys.argv:
```

```
                    print(f"\n{'='*60}")
```

```
                    print(f"[DEBUG] BREAKPOINT: HTTP 404 Error")
```

```
                    print(f"[DEBUG] URL: {url}")
```

```
                    print(f"\n{'='*60}")
```

```
                    breakpoint()
```

```
                print(f"Warning: No data for {url}")
```

```
                return None
```

```
    class MockResponse:
```

```
        def __init__(self, data):
```

```
            self._data = data
```

```
            self.status_code = 200
```

```
        def json(self):
```

```
            return self._data
```

```
        def raise_for_status(self):
```

```
            pass
```

```
        response = MockResponse(response_data)
```

```
    else:
```

```
        response = requests.get(url, timeout=timeout)
```

```

        response.raise_for_status()

    data = safe_json_parse(response)
    if not data:
        failed_requests += 1
        return None

    # Извлекаем user_id
    user_id = None
    user_data = {}

    if 'data' in data:
        if 'user' in data['data']:
            user_info = data['data']['user']
            if isinstance(user_info, dict):
                user_id = user_info.get('id')
                user_data = user_info
            elif 'id' in data['data']:
                user_id = data['data']['id']
                user_data = data['data']

    # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 3 =====
    # Остановка при отсутствии user_id
    # Запуск: python variant_11.py --debug-keys
    if user_id is None and '--debug-keys' in sys.argv:
        print(f"\n{'='*60}")
        print(f"[DEBUG] BREAKPOINT: Missing user_id")
        print(f"[DEBUG] URL: {url}")
        print(f"[DEBUG] Response: {json.dumps(data, indent=2)[:500]}")
        print(f"\n{'='*60}")
        breakpoint()

    if user_id is None:
        failed_requests += 1
        print(f"Warning: No user ID for {url}")
        return None

    successful_requests += 1
    result = {'user_id': user_id, 'data': user_data}
    api_cache.set(url, result)
    return result

except Exception as e:
    failed_requests += 1
    print(f"Error for {url}: {e}")
    return None

# ===== МАТЕМАТИЧЕСКИЕ ФУНКЦИИ =====

def calculate_ratio(a: float, b: float) -> float:
    """Вычисление соотношения с защитой от деления на ноль"""
    if b == 0:
        # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 4 =====
        # Остановка при делении на ноль
        # Запуск: python variant_11.py --debug-math
        if '--debug-math' in sys.argv:
            print(f"\n{'='*60}")

```

```

        print(f'[DEBUG] BREAKPOINT: Division by zero')
        print(f'[DEBUG] a={a}, b={b}')
        print(f'{'='*60}')
        breakpoint()
    return 0.0
return a / b

def calculate_distance(x1: float, y1: float, x2: float, y2: float) -> float:
    """Вычисление расстояния между точками"""
    dx = x2 - x1
    dy = y2 - y1
    squared = dx*dx + dy*dy
    if squared < 0:
        squared = 0
    return math.sqrt(squared)

def calculate_percentage(part: float, total: float) -> float:
    """Вычисление процента с защитой"""
    if total == 0:
        return 0.0
    return (part / total) * 100

# ===== ОБРАБОТКА ДАННЫХ =====

def process_api_data(urls: List[str]) -> List[Dict[str, Any]]:
    """
    Обработка списка URL
    Содержит условные точки останова для отладки
    """
    results = []
    total_urls = len(urls)

    print(f'\nProcessing {total_urls} URLs...')
    print("-" * 60)

    for index, url in enumerate(urls):
        # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 5 =====
        # Остановка на 5-й итерации (индекс 4)
        # Запуск: python variant_11.py --debug-specific
        if index == 4 and '--debug-specific' in sys.argv:
            print(f'\n{'='*60}')
            print(f'[DEBUG] BREAKPOINT: Specific iteration #{index}')
            print(f'[DEBUG] URL: {url}')
            print(f'[DEBUG] Results so far: {len(results)}')
            print(f'{'='*60}')
            breakpoint()

        # ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 6 =====
        # Остановка на проблемных URL
        # Запуск: python variant_11.py --debug-errors
        if ('error' in url.lower() or '404' in url) and '--debug-errors' in sys.argv:
            print(f'\n{'='*60}')
            print(f'[DEBUG] BREAKPOINT: Problematic URL')
            print(f'[DEBUG] URL: {url}')
            print(f'{'='*60}')
            breakpoint()

```

```

# ===== УСЛОВНАЯ ТОЧКА ОСТАНОВА 7 =====
# Остановка при медленных запросах
# Запуск: python variant_11.py --debug-performance
if '--debug-performance' in sys.argv:
    start_time = time.time()
    data = fetch_data(url)
    elapsed = time.time() - start_time
    if elapsed > 1.0:
        print(f"\n{'='*60}")
        print(f"[DEBUG] BREAKPOINT: Slow request")
        print(f"[DEBUG] URL: {url}")
        print(f"[DEBUG] Elapsed: {elapsed:.2f}s")
        print(f"{'='*60}")
        breakpoint()
    else:
        data = fetch_data(url)

if data and data.get('user_id'):
    processed = process_item(data)
    if processed:
        results.append(processed)

# Прогресс каждые 10 итераций
if (index + 1) % 10 == 0:
    print(f"\nProgress: {index + 1}/{total_urls}")
    stats = api_cache.get_stats()
    print(f"Cache stats:")
    print(f" Size: {stats['size']}/{stats['maxsize']}")
    print(f" Hits: {stats['hits']}")
    print(f" Misses: {stats['misses']}")
    print(f" Hit rate: {stats['hit_rate']}%")
    print(f" Total requests to cache: {stats['total_requests']}")
    print("-" * 60)

print(f"\nCompleted. Processed {len(results)} items")
return results

def process_item(data: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Обработка отдельного элемента"""
    if not data or 'user_id' not in data:
        return None

    user_id = data['user_id']
    user_data = data.get('data', {})

    return {
        'user_id': user_id,
        'name': user_data.get('name', 'Unknown'),
        'email': user_data.get('email', 'unknown@example.com'),
        'metrics': {
            'ratio': calculate_ratio(user_id, 10),
            'distance': calculate_distance(0, 0, user_id % 5, user_id // 5),
            'percentage': calculate_percentage(user_id, 100)
        }
    }

def generate_test_urls(count: int = 30) -> List[str]:

```

```

"""Генерация тестовых URL"""
urls = []
for i in range(1, count + 1):
    if i % 5 == 0:
        # Каждый 5-й URL - ошибка 404
        urls.append(f"https://httpbin.org/status/404?error={i}")
    else:
        urls.append(f"https://jsonplaceholder.typicode.com/users/{i}")
return urls

# ===== ГЛАВНАЯ ФУНКЦИЯ =====

def main():
    """Главная функция с настройкой режимов отладки"""
    global request_counter, successful_requests, failed_requests

    print("=" * 60)
    print("API Processor with Conditional Breakpoints")
    print("=" * 60)

    # Проверяем аргументы
    use_mock = '--real-api' not in sys.argv

    if '--help' in sys.argv:
        print("\nUsage: python variant_11.py [OPTIONS]")
        print("\nOPTIONS:")
        print("  --debug          Stop every 10 requests")
        print("  --debug-errors   Stop on HTTP errors and problematic URLs")
        print("  --debug-keys     Stop on missing keys in JSON")
        print("  --debug-math     Stop on mathematical errors")
        print("  --debug-specific Stop on 5th iteration")
        print("  --debug-performance Stop on slow requests (>1s)")
        print("  --real-api       Use real API (not mocks)")
        print("  --help          Show this help")
        print("\nEXAMPLES:")
        print("  python variant_11.py --debug")
        print("  python variant_11.py --debug --debug-errors")
        print("  python variant_11.py --debug-specific --debug-math")
        print("  python variant_11.py --debug --debug-errors --debug-specific")
        return

    # Активные режимы
    active_modes = [a for a in sys.argv if a.startswith('--') and a not in ['--real-api', '--help']]

    print(f"\nActive modes: {active_modes if active_modes else 'None'}")
    print(f"Using: {'MOCKS' if use_mock else 'REAL API'}")
    print("-" * 60)

    # Показываем доступные точки останова
    print("\nConditional breakpoints configured:")
    print("  1. Every 10 requests (--debug)")
    print("  2. HTTP errors (--debug-errors)")
    print("  3. Missing 'user' key (--debug-keys)")
    print("  4. Division by zero (--debug-math)")
    print("  5. 5th iteration (--debug-specific)")
    print("  6. Problematic URLs (--debug-errors)")
    print("  7. Slow requests >1s (--debug-performance)")

```

```

print("-" * 60)

# Генерируем URL
urls = generate_test_urls(30)
print(f'\nGenerated {len(urls)} test URLs')
print(f'First 5 URLs: {urls[:5]}')
print(f'Note: URLs with '404' will fail (6 URLs total)')
print("-" * 60)

# Сбрасываем счетчики
request_counter = 0
successful_requests = 0
failed_requests = 0
api_cache.cache.clear()
api_cache.hits = 0
api_cache.misses = 0
api_cache.total_requests = 0

try:
    start_time = time.time()

    # Обработываем данные
    results = process_api_data(urls)

    elapsed_time = time.time() - start_time

    # Выводим результаты
    print("\n" + "=" * 60)
    print(f'FINAL RESULTS')
    print("=" * 60)

    print(f'\nStatistics:')
    print(f' Total URLs processed: {len(urls)}')
    print(f' Successful requests: {successful_requests}')
    print(f' Failed requests: {failed_requests}')
    print(f' Items processed: {len(results)}')
    print(f' Time elapsed: {elapsed_time:.2f}s')

    cache_stats = api_cache.get_stats()
    print(f'\nCache Statistics:')
    print(f' Cache size: {cache_stats['size']}/{cache_stats['maxsize']}')
    print(f' Cache hits: {cache_stats['hits']}')
    print(f' Cache misses: {cache_stats['misses']}')
    print(f' Hit rate: {cache_stats['hit_rate']}%')
    print(f' Total requests to cache: {cache_stats['total_requests']}')

    if results:
        print(f'\nSample results (first 5):')
        for i, r in enumerate(results[:5], 1):
            print(f' {i}. User {r['user_id']}: {r['name']}')
            print(f'     Email: {r['email']}')
            print(f'     Metrics: {r['metrics']}')

    # Проверяем корректность
    expected_success = len(urls) - 6 # 6 URLs с 404
    print(f'\nVerification:')
    print(f' Expected successful: {expected_success}')

```



```

print(f" Actual successful: {successful_requests}")
if successful_requests == expected_success:
    print(" ✓ All requests processed correctly!")
else:
    print(f" X Mismatch! Expected {expected_success}, got {successful_requests}")

print("\nProgram completed successfully!")

except KeyboardInterrupt:
    print("\n\nProgram interrupted by user")
except Exception as e:
    print(f"\nError: {e}")
    import traceback
    traceback.print_exc()

# ===== ТОЧКА ВХОДА =====

if __name__ == "__main__":
    if len(sys.argv) == 1:
        print("Tip: Use --help to see available options")
        print("Example: python variant_11.py --debug")
        print("-" * 60)

    main()

```

Код 4

```
===== RESTART: Y:/24 ИСиП 2025/УП02/Группа 2/Чичерина Д/де:
💡 Tip: Use --help to see available options
📌 Example: python variant_11.py --trace-memory
-----

🔍 JSON API Processor with Memory Debugging (tracemalloc)
=====

⚙️ Active modes: None
🛠️ Using: MOCKS
-----

📋 Generated 50 test URLs
  ⓘ 10 URLs will fail (every 5th URL)

🚀 Processing 50 URLs...
⚠️ Warning: No mock data for https://httpbin.org/status/404?error=5
⚠️ Warning: No mock data for https://httpbin.org/status/404?error=10

📊 Progress: 10/50
  ⌚ Elapsed: 0.01s
  📄 Current memory: 37.21 MB
  🗄️ Cache stats:
    Size: 8/50
    Memory: 0.0021 MB
    Hit rate: 0.0%
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=15
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=20

📊 Progress: 20/50
  ⌚ Elapsed: 0.11s
  📄 Current memory: 37.22 MB
  🗄️ Cache stats:
    Size: 16/50
    Memory: 0.0041 MB
    Hit rate: 0.0%
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=25
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=30

📊 Progress: 30/50
  ⌚ Elapsed: 0.18s
  📄 Current memory: 37.23 MB
  🗄️ Cache stats:
    Size: 24/50
    Memory: 0.0062 MB
    Hit rate: 0.0%
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=35
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=40

📊 Progress: 40/50
  ⌚ Elapsed: 0.26s
  📄 Current memory: 37.25 MB
  🗄️ Cache stats:
    Size: 32/50
    Memory: 0.0082 MB
    Hit rate: 0.0%
  ⚠️ Warning: No mock data for https://httpbin.org/status/404?error=45
```

```

    Hit rate: 0.0%
    ⚠ Warning: No mock data for https://httpbin.org/status/404?error=45
    ⚠ Warning: No mock data for https://httpbin.org/status/404?error=50

    📊 Progress: 50/50
    ⌚ Elapsed: 0.34s
    📄 Current memory: 37.53 MB
    🗄 Cache stats:
        Size: 40/50
        Memory: 0.0103 MB
        Hit rate: 0.0%

    ✅ Completed in 0.39s. Processed 40 items

=====
    📊 FINAL RESULTS
=====

    📋 Statistics:
        Total requests: 50
        ✅ Successful: 40
        ❌ Failed: 10
        📄 Items processed: 40

    📄 Sample results (first 3):
        1. User 1: Leanne Graham
           Metrics: {'ratio': 0.1, 'distance': 1.0, 'percentage': 1.0}
        2. User 2: Ervin Howell
           Metrics: {'ratio': 0.2, 'distance': 2.0, 'percentage': 2.0}
        3. User 3: Clementine Bauch
           Metrics: {'ratio': 0.3, 'distance': 3.0, 'percentage': 3.0}

    🗄 Cache Statistics:
        Size: 40/50
        Hits: 0
        Misses: 50
        Hit rate: 0.0%
        Memory usage: 0.0103 MB
        Age: 0.7s

    ✅ Program completed successfully!

```

```
#!/usr/bin/env python3
```

```
"""
```

```
JSON API Processor с отладкой памяти через tracemalloc
```

```
Этап 4: Полностью исправленная и улучшенная версия
```

```
"""
```

```

import requests
import math
import time
import json
import sys
import tracemalloc
import gc
import os
from typing import Optional, Dict, List, Any, Tuple
from collections import OrderedDict

```

```

from datetime import datetime

# ===== КЛАСС КЕША С ОТСЛЕЖИВАНИЕМ ПАМЯТИ =====

class MemoryAwareCache:
    """Кеш с отслеживанием использования памяти"""

    def __init__(self, maxsize: int = 50, ttl: int = 60):
        self.cache = OrderedDict()
        self.maxsize = maxsize
        self.ttl = ttl
        self.hits = 0
        self.misses = 0
        self.total_requests = 0
        self.created_at = time.time()

    def get(self, key: str) -> Optional[Any]:
        """Получение значения из кеша"""
        self.total_requests += 1
        if key in self.cache:
            value, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                self.cache.move_to_end(key)
                self.hits += 1
                return value
            else:
                del self.cache[key]
                self.misses += 1
        self.misses += 1
        return None

    def set(self, key: str, value: Any) -> None:
        """Сохранение значения в кеше"""
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = (value, time.time())

        # Проверяем размер кеша
        if len(self.cache) > self.maxsize:
            old_key, _ = self.cache.popitem(last=False)

            # Отладочное сообщение
            if '--debug-memory' in sys.argv:
                print(f"[MEMORY DEBUG] Cache maxsize exceeded. Removed: {old_key[:50]}...")

    def clear(self) -> None:
        """Очистка кеша"""
        self.cache.clear()
        self.hits = 0
        self.misses = 0
        self.total_requests = 0

    def get_stats(self) -> Dict[str, Any]:
        """Получение статистики кеша"""
        total = self.hits + self.misses

        # Вычисляем примерный размер кеша в памяти

```

```

cache_size_bytes = 0
for key, (value, _) in self.cache.items():
    try:
        cache_size_bytes += sys.getsizeof(key)
        cache_size_bytes += sys.getsizeof(value)
    except:
        pass

return {
    'size': len(self.cache),
    'maxsize': self.maxsize,
    'hits': self.hits,
    'misses': self.misses,
    'total_requests': self.total_requests,
    'hit_rate': round((self.hits / total * 100) if total > 0 else 0, 2),
    'memory_usage_bytes': cache_size_bytes,
    'memory_usage_mb': round(cache_size_bytes / (1024 * 1024), 4),
    'age_seconds': round(time.time() - self.created_at, 1)
}

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====

api_cache = MemoryAwareCache(maxsize=50, ttl=60)
request_counter = 0
successful_requests = 0
failed_requests = 0
start_time = None

# Для tracemalloc
memory_snapshots = []
tracemalloc_enabled = False

# ===== ФУНКЦИИ ДЛЯ ОТЛАДКИ ПАМЯТИ =====

def start_memory_tracing():
    """Запуск трассировки памяти"""
    global tracemalloc_enabled
    if not tracemalloc_enabled:
        tracemalloc.start()
        tracemalloc_enabled = True
        print("[MEMORY]  Tracemalloc started")
        print(f"[MEMORY]  Initial memory usage: {get_memory_usage():.2f} MB")

def stop_memory_tracing():
    """Остановка трассировки памяти"""
    global tracemalloc_enabled
    if tracemalloc_enabled:
        tracemalloc.stop()
        tracemalloc_enabled = False
        print("[MEMORY]  Tracemalloc stopped")

def get_memory_usage() -> float:
    """Получение текущего использования памяти в МБ"""
    try:

```

```

import psutil
process = psutil.Process(os.getpid())
return process.memory_info().rss / (1024 * 1024)
except ImportError:
    # Если psutil не установлен, используем tracemalloc
    if tracemalloc_enabled:
        snapshot = tracemalloc.take_snapshot()
        total = sum(stat.size for stat in snapshot.statistics('lineno'))
        return total / (1024 * 1024)
    return 0.0

def take_memory_snapshot(name: str = ""):
    """Создание снимка памяти"""
    if not tracemalloc_enabled:
        if '--trace-memory' in sys.argv:
            print("[WARNING] ⚠ Tracemalloc not started. Use --trace-memory")
        return None

    snapshot = tracemalloc.take_snapshot()
    timestamp = time.time()
    memory_snapshots.append((name, snapshot, timestamp))

    # Анализ снимка
    print(f"\n📷 [MEMORY SNAPSHOT] {name}")
    print(f"   Time: {datetime.fromtimestamp(timestamp).strftime('%H:%M:%S')}")
    print(f"   Total snapshots: {len(memory_snapshots)}")

    # Топ-10 выделений памяти по строкам
    top_stats = snapshot.statistics('lineno')

    print(f"\n 📊 Top 10 memory allocations by line:")
    for i, stat in enumerate(top_stats[:10], 1):
        size_mb = stat.size / (1024 * 1024)
        print(f"   {i:2d}. {stat} ({size_mb:.3f} MB)")

    # Топ-5 по файлам
    top_files = snapshot.statistics('filename')
    print(f"\n 📁 Top 5 memory allocations by file:")
    for i, stat in enumerate(top_files[:5], 1):
        size_mb = stat.size / (1024 * 1024)
        print(f"   {i:2d}. {stat} ({size_mb:.3f} MB)")

    # Общая статистика
    total_memory = sum(stat.size for stat in top_stats)
    print(f"\n 📄 Total memory in snapshot: {total_memory / (1024 * 1024):.2f} MB")
    print(f"   📌 Number of allocations: {len(top_stats)}")

    return snapshot

def compare_memory_snapshots():
    """Сравнение последних двух снимков памяти"""
    if len(memory_snapshots) < 2:
        print("[MEMORY] ❗ Need at least 2 snapshots to compare")
    return

```

```

name1, snapshot1, time1 = memory_snapshots[-2]
name2, snapshot2, time2 = memory_snapshots[-1]

print(f'\n 🔄 [MEMORY COMPARISON]')
print(f' From: {datetime.fromtimestamp(time1).strftime('%H:%M:%S')} ({name1})')
print(f' To: {datetime.fromtimestamp(time2).strftime('%H:%M:%S')} ({name2})')
print(f' Duration: {time2 - time1:.2f}s')

# Сравниваем снимки
diff_stats = snapshot2.compare_to(snapshot1, 'lineno')

print(f'\n 📊 Top 10 memory changes:')
has_changes = False
total_diff = 0

for i, stat in enumerate(diff_stats[:10], 1):
    if stat.size_diff != 0:
        has_changes = True
        sign = "+" if stat.size_diff > 0 else ""
        size_kb = stat.size_diff / 1024
        total_diff += stat.size_diff
        print(f' {i:2d}. {stat} ({sign} {size_kb:.2f} KB)')

if not has_changes:
    print(" ✅ No significant memory changes detected")

# Общее изменение памяти
print(f'\n 📈 Total memory change: {total_diff / (1024 * 1024):.3f} MB')

if total_diff > 1024 * 1024: # Больше 1MB
    print(f' ⚠️ WARNING: Memory increased significantly!")
    print(f' 💡 SUGGESTION: Check for memory leaks in the code")
elif total_diff > 0:
    print(f' 📈 Memory increased slightly")
else:
    print(f' ✅ Memory usage decreased or remained stable")

def analyze_cache_memory():
    """Анализ использования памяти кешем"""
    print(f'\n 📁 [CACHE MEMORY ANALYSIS]')

    cache_stats = api_cache.get_stats()
    print(f' 📊 Cache Statistics:')
    print(f' Size: {cache_stats['size']}/{cache_stats['maxsize']}")
    print(f' Hits: {cache_stats['hits']}")
    print(f' Misses: {cache_stats['misses']}")
    print(f' Hit rate: {cache_stats['hit_rate']}%")
    print(f' Memory usage: {cache_stats['memory_usage_mb']} MB
    ({cache_stats['memory_usage_bytes']} bytes)")
    print(f' Age: {cache_stats['age_seconds']:.1f}s")

# Детальный анализ каждого элемента кеша
if cache_stats['size'] > 0:

```

```

print(f"\n 📄 Cache items detail:")
items_shown = 0
total_size = 0

for key, (value, timestamp) in list(api_cache.cache.items())[:5]:
    try:
        size = sys.getsizeof(key) + sys.getsizeof(value)
        total_size += size
        age = time.time() - timestamp
        print(f"    Key: {key[:40]}...")
        print(f"    Size: {size/1024:.2f} KB, Age: {age:.1f}s")
        items_shown += 1
    except:
        pass

if cache_stats['size'] > 5:
    print(f"    ... and {cache_stats['size'] - 5} more items")

avg_size = total_size / items_shown if items_shown > 0 else 0
print(f"\n 📊 Average item size: {avg_size/1024:.2f} KB")

```

===== МОК-ДААННЫЕ =====

```

MOCK_USERS = {
    1: {"id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz"},
    2: {"id": 2, "name": "Ervin Howell", "email": "Shanna@melissa.tv"},
    3: {"id": 3, "name": "Clementine Bauch", "email": "Nathan@yesenia.net"},
    4: {"id": 4, "name": "Patricia Lebsack", "email": "Julianne.OConner@kory.org"},
    5: {"id": 5, "name": "Chelsey Dietrich", "email": "Lucio_Hettinger@annie.ca"},
    6: {"id": 6, "name": "Mrs. Dennis Schulist", "email": "Karley_Dach@jasper.info"},
    7: {"id": 7, "name": "Kurtis Weissnat", "email": "Telly.Hoeger@billy.biz"},
    8: {"id": 8, "name": "Nicholas Runolfsson", "email": "Sherwood@rosamond.me"},
    9: {"id": 9, "name": "Glenna Reichert", "email": "Chaim_McDermott@dana.io"},
    10: {"id": 10, "name": "Clementina DuBuque", "email": "Rey.Padberg@karina.biz"},
}

```

```

def get_mock_response(url: str) -> Optional[Dict]:
    """Генерация мок-ответа для тестирования"""
    if "jsonplaceholder" in url:
        try:
            user_id = int(url.split('/')[-1])
            if user_id in MOCK_USERS:
                return {"data": {"user": MOCK_USERS[user_id]}}
            else:
                return {
                    "data": {
                        "id": user_id,
                        "name": f"User {user_id}",
                        "email": f"user{user_id}@example.com"
                    }
                }
        except ValueError:
            return None
    elif "httpbin.org/status/404" in url:
        return None

```


else:

```
    return {"data": {"user": {"id": 999, "name": "Test User", "email": "test@example.com"}}
```

===== ОСНОВНЫЕ ФУНКЦИИ =====

```
def safe_json_parse(response) -> Dict[str, Any]:
```

```
    """Безопасный парсинг JSON"""
```

```
    try:
```

```
        if hasattr(response, 'json'):
```

```
            return response.json()
```

```
        elif isinstance(response, dict):
```

```
            return response
```

```
        return {}
```

```
    except Exception as e:
```

```
        print(f"JSON parsing error: {e}")
```

```
        return {}
```

```
def fetch_data(url: str, timeout: int = 5, use_mock: bool = True) -> Optional[Dict[str, Any]]:
```

```
    """Получение данных с API с кешированием"""
```

```
    global request_counter, successful_requests, failed_requests, api_cache
```

```
    request_counter += 1
```

```
    # Проверяем кеш
```

```
    cached = api_cache.get(url)
```

```
    if cached:
```

```
        return cached
```

```
    try:
```

```
        if use_mock:
```

```
            response_data = get_mock_response(url)
```

```
            if response_data is None:
```

```
                failed_requests += 1
```

```
                print(f"⚠ Warning: No mock data for {url}")
```

```
                return None
```

```
    class MockResponse:
```

```
        def __init__(self, data):
```

```
            self._data = data
```

```
            self.status_code = 200
```

```
        def json(self):
```

```
            return self._data
```

```
        def raise_for_status(self):
```

```
            pass
```

```
        response = MockResponse(response_data)
```

```
    else:
```

```
        response = requests.get(url, timeout=timeout)
```

```
        response.raise_for_status()
```

```
    data = safe_json_parse(response)
```

```
    if not data:
```

```
        failed_requests += 1
```

```
        return None
```

```

# Извлекаем user_id
user_id = None
user_data = {}

if 'data' in data:
    if 'user' in data['data']:
        user_info = data['data']['user']
        if isinstance(user_info, dict):
            user_id = user_info.get('id')
            user_data = user_info
    elif 'id' in data['data']:
        user_id = data['data']['id']
        user_data = data['data']

if user_id is None:
    failed_requests += 1
    print(f"⚠ Warning: No user ID for {url}")
    return None

successful_requests += 1
result = {'user_id': user_id, 'data': user_data}
api_cache.set(url, result)
return result

except Exception as e:
    failed_requests += 1
    print(f"❌ Error for {url}: {e}")
    return None

```

===== МАТЕМАТИЧЕСКИЕ ФУНКЦИИ =====

```

def calculate_ratio(a: float, b: float) -> float:
    """Вычисление соотношения с защитой от деления на ноль"""
    if b == 0:
        return 0.0
    return a / b

def calculate_distance(x1: float, y1: float, x2: float, y2: float) -> float:
    """Вычисление расстояния между точками"""
    dx = x2 - x1
    dy = y2 - y1
    squared = dx*dx + dy*dy
    if squared < 0:
        squared = 0
    return math.sqrt(squared)

def calculate_percentage(part: float, total: float) -> float:
    """Вычисление процента с защитой"""
    if total == 0:
        return 0.0
    return (part / total) * 100

```

===== ОБРАБОТКА ДАННЫХ =====

```

def process_api_data(urls: List[str]) -> List[Dict[str, Any]]:
    """Обработка списка URL с отладкой памяти"""
    global start_time
    results = []
    total_urls = len(urls)

    print(f"\n🚀 Processing {total_urls} URLs...")
    start_time = time.time()

    # Начальный снимок памяти
    if '--trace-memory' in sys.argv:
        take_memory_snapshot("Start of processing")

    for index, url in enumerate(urls):
        data = fetch_data(url)

        if data and data.get('user_id') is not None:
            processed = process_item(data)
            if processed:
                results.append(processed)

    # Прогресс каждые 10 итераций
    if (index + 1) % 10 == 0:
        elapsed = time.time() - start_time
        print(f"\n📊 Progress: {index + 1}/{total_urls}")
        print(f"🕒 Elapsed: {elapsed:.2f}s")

        current_memory = get_memory_usage()
        print(f"💾 Current memory: {current_memory:.2f} MB")

        cache_stats = api_cache.get_stats()
        print(f"🗄️ Cache stats:")
        print(f"    Size: {cache_stats['size']}/{cache_stats['maxsize']}")
        print(f"    Memory: {cache_stats['memory_usage_mb']} MB")
        print(f"    Hit rate: {cache_stats['hit_rate']}%")

    # Снимок памяти каждые 10 итераций
    if '--trace-memory' in sys.argv:
        take_memory_snapshot(f"After {index + 1} requests")

    # Финальный снимок памяти
    if '--trace-memory' in sys.argv:
        take_memory_snapshot("End of processing")
        compare_memory_snapshots()

    total_time = time.time() - start_time
    print(f"\n✅ Completed in {total_time:.2f}s. Processed {len(results)} items")
    return results

def process_item(data: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Обработка отдельного элемента"""
    if not data or 'user_id' not in data:
        return None

```

```

user_id = data['user_id']
raw_data = data.get('data', {})

user_name = raw_data.get('name', 'Unknown')
user_email = raw_data.get('email', 'no-email@example.com')

# Вычисляем метрики
ratio = calculate_ratio(user_id, 10)
distance = calculate_distance(0, 0, user_id % 5, user_id // 5)
percentage = calculate_percentage(user_id, 100)

return {
    'user_id': user_id,
    'name': user_name,
    'email': user_email,
    'metrics': {
        'ratio': ratio,
        'distance': distance,
        'percentage': percentage
    }
}

```

===== ТЕСТОВЫЕ ДАННЫЕ =====

```

def generate_test_urls(count: int = 50) -> List[str]:
    """Генерация тестовых URL"""
    urls = []
    for i in range(1, count + 1):
        if i % 5 == 0:
            urls.append(f"https://httpbin.org/status/404?error={i}")
        else:
            urls.append(f"https://jsonplaceholder.typicode.com/users/{i}")
    return urls

```

===== СИМУЛЯЦИЯ УТЕЧЕК ПАМЯТИ =====

```

def simulate_memory_leak():
    """Симуляция утечки памяти для демонстрации tracemalloc"""
    print("\n🔧 [DEMO] Simulating memory leak...")
    leak_list = []

    for i in range(5):
        # Создаем большие объекты
        leak_list.append([i] * 100000)

        if i % 2 == 0:
            current_mem = get_memory_usage()
            print(f"📊 Iteration {i}: memory = {current_mem:.2f} MB")

            if '--trace-memory' in sys.argv:
                take_memory_snapshot(f"Leak iteration {i}")

    print(f"⚠️ [WARNING] {len(leak_list)} objects retained in memory!")
    print(f"💡 [SUGGESTION] These objects should be cleared when no longer needed")
    return leak_list

```

```

# ===== ГЛАВНАЯ ФУНКЦИЯ =====

def main():
    """Главная функция с отладкой памяти"""
    global api_cache, request_counter, successful_requests, failed_requests, memory_snapshots

    print("=" * 70)
    print("🔍 JSON API Processor with Memory Debugging (tracemalloc)")
    print("=" * 70)

    # Проверяем аргументы командной строки
    debug_modes = {
        '--trace-memory': 'Enable tracemalloc memory tracing',
        '--debug-memory': 'Enable memory debug messages',
        '--simulate-leak': 'Simulate memory leak for demonstration',
        '--analyze-cache': 'Analyze cache memory usage',
        '--real-api': 'Use real API (not mocks)',
        '--help': 'Show help'
    }

    active_modes = [mode for mode in debug_modes if mode in sys.argv]
    use_mock = '--real-api' not in sys.argv
    trace_memory = '--trace-memory' in sys.argv
    simulate_leak = '--simulate-leak' in sys.argv
    analyze_cache = '--analyze-cache' in sys.argv

    if '--help' in sys.argv:
        print("\n📖 Usage: python variant_11.py [OPTIONS]\n")
        print("Options:")
        for mode, desc in debug_modes.items():
            print(f"  {mode:<20} {desc}")
        print("\nExamples:")
        print("  python variant_11.py --trace-memory")
        print("  python variant_11.py --trace-memory --debug-memory")
        print("  python variant_11.py --trace-memory --simulate-leak")
        print("  python variant_11.py --analyze-cache")
        return

    print(f"\n⚙️ Active modes: {active_modes if active_modes else 'None'}")
    print(f"🛠️ Using: {'MOCKS' if use_mock else 'REAL API'}")
    print("-" * 70)

    # Сбрасываем счетчики
    request_counter = 0
    successful_requests = 0
    failed_requests = 0
    api_cache.clear()
    memory_snapshots.clear()

    # Запускаем tracemalloc
    if trace_memory:
        start_memory_tracing()
        gc.collect()
        print(f"[MEMORY] 🛠️ After GC: {get_memory_usage():.2f} MB")

```

```

# Генерируем тестовые URL
test_urls = generate_test_urls(50)
print(f'\n📄 Generated {len(test_urls)} test URLs')
failed_urls_count = sum(1 for url in test_urls if "404" in url)
print(f'❗ {failed_urls_count} URLs will fail (every 5th URL)')

try:
    # Симуляция утечки памяти
    leak_objects = None
    if simulate_leak and trace_memory:
        leak_objects = simulate_memory_leak()

    # Обработка данных
    results = process_api_data(test_urls)

    # Анализ кеша
    if analyze_cache:
        analyze_cache_memory()

    # Выводим результаты
    print("\n" + "=" * 70)
    print(f'📊 FINAL RESULTS')
    print("=" * 70)

    print(f'\n📈 Statistics:')
    print(f'   Total requests: {request_counter}')
    print(f'   ✅ Successful: {successful_requests}')
    print(f'   ❌ Failed: {failed_requests}')
    print(f'   📁 Items processed: {len(results)}')

    if results:
        print(f'\n📄 Sample results (first 3):')
        for i, result in enumerate(results[:3], 1):
            print(f'   {i}. User {result['user_id']}: {result['name']}')
            print(f'       Metrics: {result['metrics']}')

    # Статистика кеша
    cache_stats = api_cache.get_stats()
    print(f'\n📁 Cache Statistics:')
    print(f'   Size: {cache_stats['size']}/{cache_stats['maxsize']}')
    print(f'   Hits: {cache_stats['hits']}')
    print(f'   Misses: {cache_stats['misses']}')
    print(f'   Hit rate: {cache_stats['hit_rate']}%')
    print(f'   Memory usage: {cache_stats['memory_usage_mb']} MB')
    print(f'   Age: {cache_stats['age_seconds']:.1f}s')

    # Финальный анализ памяти
    if trace_memory:
        print("\n" + "-" * 70)
        print(f'📄 FINAL MEMORY ANALYSIS')
        print("-" * 70)

        final_memory = get_memory_usage()
        print(f'   Final memory usage: {final_memory:.2f} MB')

```

```

# Проверяем наличие утечек
if simulate_leak and leak_objects:
    print(f"\n ⚠️ [WARNING] {len(leak_objects)} objects still in memory!")
    print(f"💡 [SUGGESTION] Use tracemalloc to find where objects are created")

print("\n💡 MEMORY RECOMMENDATIONS:")
print(" 1. Use cache with TTL to limit memory growth")
print(" 2. Clear large objects when no longer needed")
print(" 3. Use generators instead of lists for large datasets")
print(" 4. Profile memory with tracemalloc regularly")

# Останавливаем tracemalloc
if trace_memory:
    stop_memory_tracing()

print("\n✅ Program completed successfully!")

except KeyboardInterrupt:
    print("\n🛑 Program interrupted by user")
except Exception as e:
    print(f"\n❌ Error: {e}")
    import traceback
    traceback.print_exc()
finally:
    if trace_memory and tracemalloc_enabled:
        stop_memory_tracing()

if __name__ == "__main__":
    if len(sys.argv) == 1:
        print("💡 Tip: Use --help to see available options")
        print("🔗 Example: python variant_11.py --trace-memory")
        print("-" * 70)

    main()

```

Код 5

```
=====
API Processor - Fixed Logic (Stage 5)
Safety: Division by Zero, Negative Sqrt, etc.
=====

Using: MOCKS

Generated 30 test URLs
6 URLs will fail

Active Safety Features:
- [x] Division by zero protection
- [x] Negative sqrt protection
- [x] BMI validation
- [x] Age/Salary validation
- [x] Performance classification

13:32:09 - INFO - Processing 30 URLs...
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=5
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=10
13:32:09 - INFO - Progress: 10/30
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=15
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=20
13:32:09 - INFO - Progress: 20/30
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=25
13:32:09 - WARNING - No mock data for https://httpbin.org/status/404?error=30
13:32:09 - INFO - Progress: 30/30
13:32:09 - INFO - Completed. Processed 24 items

RESULTS

Statistics:
Total requests: 30
- [x] Successful: 24
- [x] Failed: 6
- [x] Items processed: 24
- [x] Time: 0.08s

Sample results (first 3):
1. User 1: Leanne Graham
   Age: 28, Salary: 50000.0
   BMI: 22.9 (Normal weight)
   Performance: 0.75 (High)
2. User 2: Ervin Howell
   Age: 32, Salary: 60000.0
   BMI: 23.1 (Normal weight)
   Performance: 0.82 (Excellent)
3. User 3: Clementine Bauch
   Age: 25, Salary: 45000.0
   BMI: 22.0 (Normal weight)
   Performance: 0.72 (High)

Cache:
Size: 24/50
Hit rate: 0.0%

[x] Expected: 24, Got: 24
```

```
#!/usr/bin/env python3
```

```
"""
```

JSON API Processor с исправлением логики

Этап 5: Полностью исправленная версия с защитой от всех ошибок

```
"""
```

```
import requests
```

```
import math
```

```
import time
```

```
import json
```

```
import sys
```

```
from typing import Optional, Dict, List, Any
```

```
from dataclasses import dataclass, field
```

```
from enum import Enum
```

```
import logging
```

```
# ===== НАСТРОЙКА ЛОГГИРОВАНИЯ =====
```

```
logging.basicConfig(
```

```
    level=logging.INFO,
```

```
    format='%(asctime)s - %(levelname)s - %(message)s',
```

```
    datefmt='%H:%M:%S'
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```
# ===== ENUM ДЛЯ КАТЕГОРИЙ =====
```



```

class BMICategory(Enum):
    UNDERWEIGHT = "Underweight"
    NORMAL = "Normal weight"
    OVERWEIGHT = "Overweight"
    OBESE = "Obese"
    UNKNOWN = "Unknown"

class PerformanceLevel(Enum):
    LOW = "Low"
    MEDIUM = "Medium"
    HIGH = "High"
    EXCELLENT = "Excellent"
    UNKNOWN = "Unknown"

# ===== DATA CLASSES =====

@dataclass
class BMICalculation:
    bmi: float = 0.0
    category: str = "Unknown"
    valid: bool = False
    error: Optional[str] = None

@dataclass
class UserMetrics:
    bmi: Optional[float] = None
    bmi_category: str = "Unknown"
    age_score: float = 0.0
    salary_ratio: float = 0.0
    performance_score: float = 0.0
    performance_level: str = "Unknown"
    valid: bool = False
    errors: List[str] = field(default_factory=list)

@dataclass
class ProcessedUser:
    user_id: int
    name: str
    email: str
    age: int
    salary: float
    metrics: UserMetrics
    warnings: List[str] = field(default_factory=list)

# ===== KJIACC KEIIIA =====

class SimpleCache:
    def __init__(self, maxsize: int = 50):
        self.cache = {}
        self.maxsize = maxsize
        self.hits = 0
        self.misses = 0
        self.total_requests = 0

    def get(self, key: str) -> Optional[Any]:
        self.total_requests += 1
        if key in self.cache:

```

```

        self.hits += 1
        return self.cache[key]
    self.misses += 1
    return None

def set(self, key: str, value: Any) -> None:
    if len(self.cache) >= self.maxsize:
        first_key = next(iter(self.cache))
        del self.cache[first_key]
    self.cache[key] = value

def get_stats(self) -> Dict[str, Any]:
    total = self.hits + self.misses
    return {
        'size': len(self.cache),
        'maxsize': self.maxsize,
        'hits': self.hits,
        'misses': self.misses,
        'total_requests': self.total_requests,
        'hit_rate': round((self.hits / total * 100) if total > 0 else 0, 2)
    }

# ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====

api_cache = SimpleCache(maxsize=50)
request_counter = 0
successful_requests = 0
failed_requests = 0
all_warnings = []
all_errors = []

# ===== МОК-ДААННЫЕ =====

MOCK_USERS = {
    1: {"id": 1, "name": "Leanne Graham", "email": "Sincere@april.biz",
        "age": 28, "salary": 50000, "weight": 70, "height": 1.75},
    2: {"id": 2, "name": "Ervin Howell", "email": "Shanna@melissa.tv",
        "age": 32, "salary": 60000, "weight": 75, "height": 1.80},
    3: {"id": 3, "name": "Clementine Bauch", "email": "Nathan@yesenia.net",
        "age": 25, "salary": 45000, "weight": 60, "height": 1.65},
    4: {"id": 4, "name": "Patricia Lebsack", "email": "Julianne.OConner@kory.org",
        "age": 40, "salary": 75000, "weight": 80, "height": 1.70},
    5: {"id": 5, "name": "Chelsey Dietrich", "email": "Lucio_Hettinger@annic.ca",
        "age": 35, "salary": 0, "weight": 65, "height": 1.68},
    6: {"id": 6, "name": "Mrs. Dennis Schulist", "email": "Karley_Dach@jasper.info",
        "age": 29, "salary": 55000, "weight": 68, "height": 1.72},
    7: {"id": 7, "name": "Kurtis Weissnat", "email": "Telly.Hoeger@billy.biz",
        "age": 42, "salary": 80000, "weight": 85, "height": 1.78},
    8: {"id": 8, "name": "Nicholas Runolfsson", "email": "Sherwood@rosamond.me",
        "age": 0, "salary": 65000, "weight": 72, "height": 1.75},
    9: {"id": 9, "name": "Glenna Reichert", "email": "Chaim_McDermott@dana.io",
        "age": 55, "salary": 90000, "weight": 78, "height": 1.65},
    10: {"id": 10, "name": "Clementina DuBuque", "email": "Rey.Padberg@karina.biz",
        "age": 30, "salary": 70000, "weight": 62, "height": 1.60},
}

def get_mock_response(url: str) -> Optional[Dict]:

```

```

"""Генерация мок-ответа"""
if "jsonplaceholder" in url:
    try:
        user_id = int(url.split('/')[-1])
        if user_id in MOCK_USERS:
            return {"data": {"user": MOCK_USERS[user_id]}}
        else:
            return {
                "data": {
                    "user": {
                        "id": user_id,
                        "name": f"User {user_id}",
                        "email": f"user{user_id}@example.com",
                        "age": 20 + (user_id % 50),
                        "salary": 30000 + (user_id * 1000),
                        "weight": 60 + (user_id % 30),
                        "height": 1.60 + ((user_id % 20) / 100)
                    }
                }
            }
        except ValueError:
            return None
    elif "httpbin.org/status/404" in url:
        return None
    else:
        return {
            "data": {
                "user": {
                    "id": 999,
                    "name": "Test User",
                    "email": "test@example.com",
                    "age": 25,
                    "salary": 50000,
                    "weight": 70,
                    "height": 1.75
                }
            }
        }

```

===== БЕЗОПАСНЫЕ МАТЕМАТИЧЕСКИЕ ФУНКЦИИ =====

```
def safe_division(numerator: float, denominator: float, default: float = 0.0) -> float:
```

```
    """Безопасное деление"""
```

```
    if denominator == 0 or abs(denominator) < 1e-308:
```

```
        return default
```

```
    if math.isinf(numerator) or math.isinf(denominator):
```

```
        return default
```

```
    if math.isnan(numerator) or math.isnan(denominator):
```

```
        return default
```

```
    try:
```

```
        result = numerator / denominator
```

```
        return default if (math.isinf(result) or math.isnan(result)) else result
```

```
    except:
```

```
        return default
```

```
def safe_sqrt(value: float, default: float = 0.0) -> float:
```

```
    """Безопасное извлечение корня"""
```

```

if value < 0:
    value = abs(value)
if math.isinf(value) or math.isnan(value):
    return default
try:
    result = math.sqrt(value)
    return default if math.isnan(result) else result
except:
    return default

def safe_round(value: float, decimals: int = 2) -> float:
    """Безопасное округление"""
    if math.isnan(value) or math.isinf(value):
        return 0.0
    try:
        return round(value, decimals)
    except:
        return value

def clamp(value: float, min_val: float, max_val: float) -> float:
    """Ограничение значения"""
    if math.isnan(value) or math.isinf(value):
        return min_val
    return max(min_val, min(value, max_val))

# ===== ФУНКЦИИ РАСЧЕТА =====

def calculate_bmi(weight: float, height: float) -> BMICalculation:
    """Расчет ИМТ"""
    if weight <= 0:
        return BMICalculation(valid=False, error=f"Invalid weight: {weight}")
    if height <= 0:
        return BMICalculation(valid=False, error=f"Invalid height: {height}")

    weight = min(weight, 500.0)
    height = min(height, 3.0)

    height_squared = height * height
    if height_squared <= 0:
        return BMICalculation(valid=False, error="Height squared is zero")

    bmi = safe_division(weight, height_squared, 0.0)
    if bmi <= 0:
        return BMICalculation(valid=False, error=f"Invalid BMI: {bmi}")

    if bmi < 18.5:
        category = BMICategory.UNDERWEIGHT.value
    elif bmi < 25:
        category = BMICategory.NORMAL.value
    elif bmi < 30:
        category = BMICategory.OVERWEIGHT.value
    else:
        category = BMICategory.OBESE.value

    return BMICalculation(bmi=safe_round(bmi, 1), category=category, valid=True)

def calculate_user_metrics(user_data: Dict[str, Any]) -> UserMetrics:

```

```

"""Расчет метрик пользователя"""
errors = []

age = user_data.get('age', 25)
salary = user_data.get('salary', 50000)
weight = user_data.get('weight', 70)
height = user_data.get('height', 1.75)

# BMI
bmi_result = calculate_bmi(weight, height)
bmi_value = bmi_result.bmi if bmi_result.valid else None
bmi_category = bmi_result.category if bmi_result.valid else BMICategory.UNKNOWN.value
if not bmi_result.valid and bmi_result.error:
    errors.append(f'BMI Error: {bmi_result.error}')

# Age score
if age < 0:
    errors.append(f'Negative age: {age}')
    age = abs(age)
if age > 150:
    errors.append(f'Suspicious age: {age}')
    age = 150
age_score = clamp(safe_division(age, 100, 0.0), 0.0, 1.0)

# Salary ratio
if salary < 0:
    errors.append(f'Negative salary: {salary}')
    salary = abs(salary)
salary_ratio = clamp(safe_division(salary, 100000, 0.0), 0.0, 2.0)

# Performance score
components = []

if bmi_value is not None and 0 < bmi_value < 50:
    bmi_score = clamp(1 - abs(bmi_value - 22) / 22, 0.0, 1.0)
    components.append(bmi_score)

age_performance = clamp(1 - abs(age_score - 0.35) / 0.35, 0.0, 1.0)
components.append(age_performance)

salary_performance = min(1.0, salary_ratio)
components.append(salary_performance)

performance_score = sum(components) / len(components) if components else 0.0
performance_score = clamp(performance_score, 0.0, 1.0)

# Performance level
if performance_score >= 0.8:
    level = PerformanceLevel.EXCELLENT.value
elif performance_score >= 0.6:
    level = PerformanceLevel.HIGH.value
elif performance_score >= 0.4:
    level = PerformanceLevel.MEDIUM.value
else:
    level = PerformanceLevel.LOW.value

return UserMetrics(

```

```

        bmi=bmi_value,
        bmi_category=bmi_category,
        age_score=safe_round(age_score, 2),
        salary_ratio=safe_round(salary_ratio, 2),
        performance_score=safe_round(performance_score, 2),
        performance_level=level,
        valid=bool(components),
        errors=errors
    )

```

===== ОСНОВНЫЕ ФУНКЦИИ =====

```
def safe_json_parse(response) -> Dict[str, Any]:
```

```
    """Безопасный парсинг JSON"""
```

```
    try:
```

```
        if hasattr(response, 'json'):
```

```
            return response.json()
```

```
        elif isinstance(response, dict):
```

```
            return response
```

```
        return {}
```

```
    except:
```

```
        return {}
```

```
def fetch_data(url: str, timeout: int = 5, use_mock: bool = True) -> Optional[Dict[str, Any]]:
```

```
    """Получение данных"""
```

```
    global request_counter, successful_requests, failed_requests
```

```
    request_counter += 1
```

```
    cached = api_cache.get(url)
```

```
    if cached:
```

```
        return cached
```

```
    try:
```

```
        if use_mock:
```

```
            response_data = get_mock_response(url)
```

```
            if response_data is None:
```

```
                failed_requests += 1
```

```
                logger.warning(f"No mock data for {url}")
```

```
                return None
```

```
    class MockResponse:
```

```
        def __init__(self, data):
```

```
            self._data = data
```

```
            self.status_code = 200
```

```
        def json(self):
```

```
            return self._data
```

```
        def raise_for_status(self):
```

```
            pass
```

```
        response = MockResponse(response_data)
```

```
    else:
```

```
        response = requests.get(url, timeout=timeout)
```

```
        response.raise_for_status()
```

```
    data = safe_json_parse(response)
```

```
    if not data:
```

```

        failed_requests += 1
        return None

    user_id = None
    user_data = {}

    if 'data' in data:
        if 'user' in data['data']:
            user_info = data['data']['user']
            if isinstance(user_info, dict):
                user_id = user_info.get('id')
                user_data = user_info
            elif 'id' in data['data']:
                user_id = data['data']['id']
                user_data = data['data']

    if user_id is None:
        failed_requests += 1
        return None

    successful_requests += 1
    result = {'user_id': user_id, 'data': user_data}
    api_cache.set(url, result)
    return result

except Exception as e:
    failed_requests += 1
    logger.error(f'Error for {url}: {e}')
    return None

def process_item(data: Dict[str, Any]) -> Optional[ProcessedUser]:
    """Обработка элемента"""
    global all_warnings, all_errors

    if not data or 'user_id' not in data:
        return None

    user_id = data['user_id']
    user_data = data.get('data', {})

    name = user_data.get('name', 'Unknown')
    email = user_data.get('email', 'unknown@example.com')
    age = user_data.get('age', 25)
    salary = user_data.get('salary', 50000)
    warnings = []

    if age < 0:
        warnings.append(f'Negative age: {age}')
        age = abs(age)
    if age > 150:
        warnings.append(f'Suspicious age: {age}')
        age = 150

    if salary < 0:
        warnings.append(f'Negative salary: {salary}')
        salary = abs(salary)

```

```

metrics = calculate_user_metrics(user_data)

for error in metrics.errors:
    all_errors.append(f"User {user_id}: {error}")

return ProcessedUser(
    user_id=user_id,
    name=name,
    email=email,
    age=age,
    salary=float(salary),
    metrics=metrics,
    warnings=warnings
)

def process_api_data(urls: List[str]) -> List[ProcessedUser]:
    """Обработка списка URL"""
    results = []
    total_urls = len(urls)

    logger.info(f"Processing {total_urls} URLs...")

    for index, url in enumerate(urls):
        data = fetch_data(url)
        if data and data.get('user_id') is not None:
            processed = process_item(data)
            if processed:
                results.append(processed)

        if (index + 1) % 10 == 0:
            logger.info(f"Progress: {index + 1}/{total_urls}")

    logger.info(f"Completed. Processed {len(results)} items")
    return results

def generate_test_urls(count: int = 30) -> List[str]:
    """Генерация тестовых URL"""
    urls = []
    for i in range(1, count + 1):
        if i % 5 == 0:
            urls.append(f"https://httpbin.org/status/404?error={i}")
        else:
            urls.append(f"https://jsonplaceholder.typicode.com/users/{i}")
    return urls

# ===== ТЕСТЫ =====

def run_tests():
    """Запуск тестов"""
    print("\n" + "=" * 60)
    print("🔧 RUNNING TESTS")
    print("=" * 60)

    # Тест safe_division
    print("\n🇷🇺 safe_division:")
    for num, den in [(10, 2), (10, 0), (-10, 2), (float('inf'), 2)]:
        result = safe_division(num, den)

```



```

    print(f" {num}/{den} = {result}")

# Тест safe_sqrt
print("\n🇮🇹 safe_sqrt:")
for val in [25, -16, float('inf')]:
    result = safe_sqrt(val)
    print(f" sqrt({val}) = {result:.2f}")

# Тест calculate_bmi
print("\n🇮🇹 calculate_bmi:")
for weight, height in [(70, 1.75), (0, 1.75), (70, 0)]:
    result = calculate_bmi(weight, height)
    if result.valid:
        print(f" BMI({weight}kg, {height}m) = {result.bmi} ({result.category})")
    else:
        print(f" BMI({weight}kg, {height}m) = Error: {result.error}")

print("\n" + "=" * 60)
print("✅ Tests completed")
print("=" * 60)

# ===== ГЛАВНАЯ ФУНКЦИЯ =====

def main():
    """Главная функция"""
    global all_warnings, all_errors

    print("=" * 70)
    print("🔧 API Processor - Fixed Logic (Stage 5)")
    print("🛡️ Safety: Division by Zero, Negative Sqrt, etc.")
    print("=" * 70)

    if '--test' in sys.argv:
        run_tests()
        return

    if '--help' in sys.argv:
        print("\nUsage: python variant_11.py [OPTIONS]")
        print("\nOptions:")
        print("  --test      Run safety tests")
        print("  --real-api   Use real API (not mocks)")
        print("  --help      Show this help")
        return

    use_mock = '--real-api' not in sys.argv

    print(f"\n🐞 Using: {'MOCKS' if use_mock else 'REAL API'}")
    print("-" * 70)

    # Сброс счетчиков
    global request_counter, successful_requests, failed_requests
    request_counter = 0
    successful_requests = 0
    failed_requests = 0
    all_warnings = []
    all_errors = []

```

```
api_cache.cache.clear()
```

```
test_urls = generate_test_urls(30)
print(f'\n📄 Generated {len(test_urls)} test URLs')
failed_urls = sum(1 for url in test_urls if "404" in url)
print(f'❗ {failed_urls} URLs will fail')
```

```
print("\n🛡️ Active Safety Features:")
print("✅ Division by zero protection")
print("✅ Negative sqrt protection")
print("✅ BMI validation")
print("✅ Age/Salary validation")
print("✅ Performance classification")
print("-" * 70)
```

try:

```
start_time = time.time()
results = process_api_data(test_urls)
elapsed = time.time() - start_time
```

```
print("\n" + "=" * 70)
print("📊 RESULTS")
print("=" * 70)
```

```
print(f'\n📈 Statistics:')
print(f'   Total requests: {request_counter}')
print(f'   ✅ Successful: {successful_requests}')
print(f'   ❌ Failed: {failed_requests}')
print(f'   📁 Items processed: {len(results)}')
print(f'   ⌚ Time: {elapsed:.2f}s')
```

if results:

```
print(f'\n📄 Sample results (first 3):')
for i, result in enumerate(results[:3], 1):
    print(f'   {i}. User {result.user_id}: {result.name}')
    print(f'   Age: {result.age}, Salary: {result.salary}')
    if result.metrics.bmi:
        print(f'   BMI: {result.metrics.bmi} ({result.metrics.bmi_category})')
        print(f'   Performance: {result.metrics.performance_score} '
              f'({result.metrics.performance_level})')
```

```
cache_stats = api_cache.get_stats()
print(f'\n📁 Cache:')
print(f'   Size: {cache_stats['size']}/{cache_stats['maxsize']}')
print(f'   Hit rate: {cache_stats['hit_rate']}%')
```

```
expected = len(test_urls) - failed_urls
print(f'\n✅ Expected: {expected}, Got: {successful_requests}')
```

except KeyboardInterrupt:

```
print("\n🛑 Interrupted")
```

except Exception as e:

```
print(f'\n❌ Error: {e}')
```

```
if __name__ == "__main__":  
    if len(sys.argv) == 1:  
        print("💡 Tip: Use --help for options")  
        print("🔗 Example: python variant_11.py --test")  
        print("-" * 70)  
    main()
```

Что было сделано:

1. Выявлены и исправлены ошибки в коде:

- **KeyError: 'user'** - отсутствие ключа в JSON ответе API
- **Утечка памяти** - кеш без ограничения размера
- **Деление на ноль** - добавлена защита
- **Корень из отрицательного числа** - добавлена защита

2. Использованы инструменты отладки:

- **pdb** - условные точки останова, просмотр переменных
- **tracemalloc** - отслеживание утечек памяти

3. Созданы безопасные функции:

- **safe_division()** - защита от деления на ноль
- **safe_sqrt()** - защита от отрицательных чисел
- **safe_percentage()** - безопасный расчет процентов

Вывод: Я изучила способы отладки программы с использованием языка программирования Python.

Контрольные вопросы:

A1. Что такое стек вызовов (call stack) в Python и как его увидеть?

Стек вызовов — это последовательность вложенных функций от точки входа до места ошибки, которую можно увидеть в traceback при возникновении исключения или в pdb с помощью команды where.

A2. В чем разница между except: и except Exception as e:?

Пустой `except` перехватывает абсолютно все исключения, включая системные, что считается антипаттерном, а `except Exception as e` перехватывает только обычные ошибки и даёт доступ к объекту ошибки для вывода информации.

А3. Как запустить Python-скрипт под отладчиком `pdb` без изменения исходного кода?

Для запуска скрипта под отладчиком без изменения кода используется команда `python -m pdb script.py` в командной строке.

А4. Какие основные команды `pdb` вы знаете и для чего они нужны?

Основные команды `pdb`: `n` для выполнения следующей строки, `s` для входа в функцию, `c` для продолжения до следующей точки останова, `p` и `pp` для печати переменных, `l` для показа кода, `w` для просмотра стека и `q` для выхода из отладчика.

А5. Как установить точку останова (breakpoint) в `pdb` на определенной строке или функции?

Точку останова можно установить в коде через `breakpoint()` или `import pdb; pdb.set_trace()`, а в `pdb` интерактивно через команду `break` с указанием номера строки или имени функции, например `break 42` или `break process_booking`.

А6. Что такое утечка памяти в Python и как она может возникнуть?

Утечка памяти — это ситуация, когда программа не освобождает память после использования, и её потребление постоянно растёт из-за глобальных кешей без ограничения, замыканий, не закрытых файлов или ссылок на объекты в циклах.

А7. Какой модуль в Python позволяет отслеживать выделение памяти?

Для отслеживания выделения памяти используется встроенный модуль `tracemalloc`, который позволяет делать снимки памяти и сравнивать их для выявления утечек.

А8. Как вывести красивый `traceback` ошибки, чтобы понять, где именно произошло исключение?

Красивый `traceback` выводится с помощью `traceback.print_exc()` в блоке `except`, который показывает полную информацию об ошибке с номерами строк и путём выполнения.

А9. Что такое `RecursionError` и как его избежать?

`RecursionError` возникает при превышении лимита рекурсии (по умолчанию 1000) и избегается путём увеличения лимита через `sys.setrecursionlimit()` или заменой рекурсии на итеративный алгоритм.

A10. Для чего нужна команда `up` в `pdb`?

Команда `up` в `pdb` нужна для перемещения на уровень выше по стеку вызовов, что позволяет исследовать переменные в родительской функции, вызвавшей текущую.

B1. В чем разница между `breakpoint()` и `import pdb; pdb.set_trace()`?

`Breakpoint()` — это современный способ для Python 3.7 и выше, который короче и может быть отключён переменной окружения, а `pdb.set_trace()` работает во всех версиях, но всегда вызывает именно `pdb` и не может быть отключён.

B2. Как в `pdb` продолжить выполнение до того момента, когда некоторая переменная станет равной определенному значению?

Для продолжения выполнения до изменения переменной нужно установить условную точку останова через `break` с условием, например `break 42 if x == 100`, после чего использовать команду `c` для продолжения до выполнения условия.

B3. Почему использование `print()` для отладки считается непрофессиональным и к каким проблемам это может привести?

Использование `print()` непрофессионально, потому что это создаёт мусор в консоли, не гибко в настройке, не имеет уровней важности, остаётся в продакшене и сложно анализировать, в отличие от логирования, которое позволяет фильтровать сообщения по уровням и сохранять их в файлы.

B4. Как использовать `tracemalloc` для поиска утечек памяти в долгоживущем приложении?

Для поиска утечек памяти `tracemalloc` запускается в начале работы, затем периодически делаются снимки памяти через `take_snapshot`, которые сравниваются между собой методом `compare_to` для выявления строк кода, создающих больше всего новых объектов, и при обнаружении утечек добавляется очистка кеша или ограничение его размера.